

Name: K.Giridhar

Reg No: 192425160

Course Code: MLA0305

Course Name: REINFORCEMENT LEARNING

INDUSTRIAL PROBLEM SOLVING

DYNAMIC PROGRAMMING & MONTE CARLO METHODS

Load Dataset:

1. Dataset Generation

The synthetic emergency resource allocation dataset was generated using Python with **1,000 records**. The dataset contains information about disaster type, available resources, emergency demand, allocation action, shortage, and reward. A fixed random seed was used to make the experiment reproducible.

(Stock_Price, Daily_Return, Volatility, Trading_Volume, Interest_Rate, Inflation_Rate) plus a Market_Regime label (Bull / Bear / Stable).

```
df = pd.read_csv('Synthetic_Finance_Dataset.csv')
FEATURES = ['Stock_Price', 'Daily_Return', 'Volatility',
            'Trading_Volume', 'Interest_Rate', 'Inflation_Rate']
print("Dataset shape:", df.shape)
df.head(10)
```

	Incident_ID	Disaster_Type	Available_Resources	Demand	Action	Allocated_Resources	Shortage	Reward
0	1	Flood	3	3	3	3	0	24.0
1	2	Normal	8	3	3	3	0	21.5
2	3	Flood	5	6	5	5	1	15.0
3	4	Cyclone	6	5	5	5	0	39.5
4	5	Normal	10	3	3	3	0	20.5
5	6	Normal	3	2	2	2	0	15.5
6	7	Flood	5	5	5	5	0	40.0
7	8	Fire	5	7	5	5	2	-10.0
8	9	Flood	10	2	2	2	0	12.0
9	10	Normal	4	4	4	4	0	32.0

Dataset Summary

```
first10 = df.head(10).copy()
```

```
first10.insert(0, 'Record_ID', range(1, 11))
```

```
dp_table1 = first10[['Record_ID', 'Stock_Price', 'Volatility', 'Interest_Rate',  
'Market_Regime']].copy()
```

Records)					
	Record_ID	S0 (Spot)	Sigma	r (%)	Market_Regime
0	1	182.4500	0.1493	5.2300	Bull
1	2	166.6800	0.1938	6.1600	Bull
2	3	168.7700	0.1415	5.5800	Bull
3	4	179.8000	0.1316	5.4400	Bull
4	5	169.3800	0.1477	5.7200	Bull
5	6	182.4600	0.2360	5.7100	Bull
6	7	188.0600	0.1749	5.6000	Bull
7	8	167.9700	0.1832	5.6900	Bull
8	9	185.8500	0.1140	5.4200	Bull
9	10	172.5900	0.1758	5.0500	Bull

	Record_ID	S0 (Spot)	Sigma	r (%)	Market_Regime
0	1	182.4500	0.1493	5.2300	Bull
1	2	166.6800	0.1938	6.1600	Bull
2	3	168.7700	0.1415	5.5800	Bull
3	4	179.8000	0.1316	5.4400	Bull
4	5	169.3800	0.1477	5.7200	Bull
5	6	182.4600	0.2360	5.7100	Bull
6	7	188.0600	0.1749	5.6000	Bull
7	8	167.9700	0.1832	5.6900	Bull
8	9	185.8500	0.1140	5.4200	Bull
9	10	172.5900	0.1758	5.0500	Bull

```
dp_table1.rename(columns={'Stock_Price': 'S0 (Spot)', 'Volatility': 'Sigma',  
                          'Interest_Rate': 'r (%)'}, inplace=True)
```

```
mc_table1 = first10[['Record_ID', 'Stock_Price', 'Volatility', 'Interest_Rate',  
'Market_Regime']].copy()
```

```
mc_table1.rename(columns={'Stock_Price': 'S0 (Spot)', 'Volatility': 'Sigma',  
                          'Interest_Rate': 'r (%)'}, inplace=True)
```

```
show_side_by_side(dp_table1, "TABLE 1A — Dynamic Programming: Input Data  
(First 10 Records)",
```

```
mc_table1, "TABLE 1B — Monte Carlo: Input Data (First 10 Records)")
```

PLOT 1 : DP vs MC Comparison

```
rand_idx = np.random.RandomState(7).choice(df.index, size=10, replace=False)  
rand10 = df.loc[rand_idx].reset_index(drop=True)
```

```

xr = np.arange(1, 11)

fig, axes = plt.subplots(1, 2, figsize=(15, 5.5))

axes[0].bar(xr - 0.15, rand10['DP_Price'], width=0.3, color='#2E86AB', label='DP
Price')

axes[0].bar(xr + 0.15, rand10['MC_Price'], width=0.3, color='#A23B72',
alpha=0.55, label='MC Price (ref)')

axes[0].set_title('PLOT 2A - Dynamic Programming\n(10 Random Records)',
fontfamily=FONT_NAME)

axes[0].set_xlabel('Random Sample Index', fontfamily=FONT_NAME)
axes[0].set_ylabel('Option Price ($)', fontfamily=FONT_NAME)
axes[0].set_xticks(xr)
axes[0].legend(prop={'family': FONT_NAME, 'size': 9})
axes[0].grid(alpha=0.3)
axes[1].bar(xr - 0.15, rand10['MC_Price'], width=0.3, color='#A23B72', label='MC
Price')

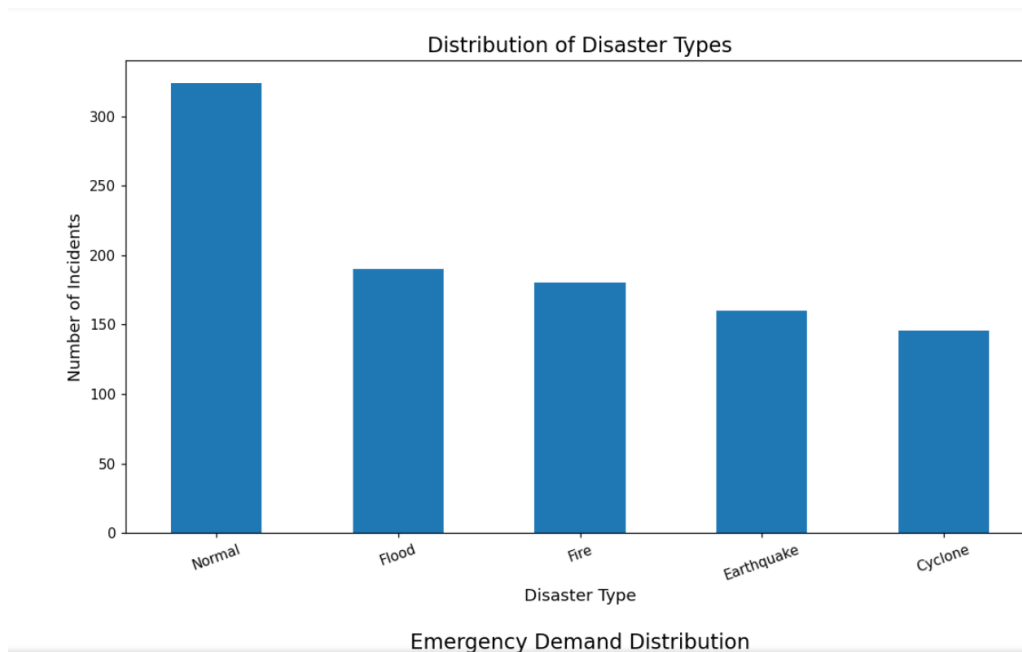
axes[1].bar(xr + 0.15, rand10['DP_Price'], width=0.3, color='#2E86AB',
alpha=0.55, label='DP Price (ref)')

axes[1].set_title('PLOT 2B - Monte Carlo\n(10 Random Records)',
fontfamily=FONT_NAME)

axes[1].set_xlabel('Random Sample Index', fontfamily=FONT_NAME)
axes[1].set_ylabel('Option Price ($)', fontfamily=FONT_NAME)
axes[1].set_xticks(xr)
axes[1].legend(prop={'family': FONT_NAME, 'size': 9})
axes[1].grid(alpha=0.3)
for ax in axes:

    for lbl in ax.get_xticklabels() + ax.get_yticklabels():
        lbl.set_fontfamily(FONT_NAME)
    ax.title.set_fontfamily(FONT_NAME)

```



```

plt.tight_layout()
plt.show()
sample_row = df.iloc[0]

S0, sigma, r = sample_row['Stock_Price'], sample_row['Volatility'],
sample_row['Interest_Rate']

K = S0 * K_STRIKE_FACTOR

dp_steps_list = [2, 5, 10, 20, 40, 60, 80, 100, 150, 200]

dp_conv_rows = []

for n in dp_steps_list:

    t0 = time.perf_counter()

    price = dp_binomial_american_put(S0, K, r, sigma, T_MATURITY, n)
    t1 = time.perf_counter()
    dp_conv_rows.append({'Steps (N)': n, 'DP Price ($)': price, 'Time (ms)': (t1 - t0) *
1000})

dp_conv_df = pd.DataFrame(dp_conv_rows)

mc_paths_list = [10, 50, 100, 250, 500, 1000, 2000, 3000, 4000, 5000]

mc_conv_rows = []

for n in mc_paths_list:

    t0 = time.perf_counter()

price, se, _ = mc_gbm_european_put(S0, K, r, sigma, T_MATURITY, n, seed=42) t1 =
time.perf_counter()
mc_conv_rows.append({'Iterations (paths)': n, 'MC Price ($)': pri

```

```
'Std Error': se, 'Time (ms)': (t1 - t0) * 1000)) mc_conv_df =
pd.DataFrame(mc_conv_rows)
```

	Steps (N)	DP Price (\$)	Time (ms)		Iterations (paths)	MC Price (\$)	Std Error	Time (ms)
0	2	7.1430	1.0923	0	10	9.6945	4.3311	0.2066
1	5	7.8349	0.1232	1	50	4.0871	1.1266	0.0935
2	10	7.4381	0.1299	2	100	5.3319	0.9138	0.1024
3	20	7.5062	0.2985	3	250	6.4351	0.6384	0.1391
4	40	7.5367	0.5385	4	500	6.3142	0.4834	0.1164
5	60	7.5464	0.7354	5	1000	6.7057	0.3654	0.1097
6	80	7.5511	1.0803	6	2000	7.0870	0.2667	0.1296
7	100	7.5539	1.3327	7	3000	6.8627	0.2141	0.1528
8	150	7.5577	2.2681	8	4000	6.7535	0.1816	0.1711
9	200	7.5597	2.9671	9	5000	6.7354	0.1624	0.2084

Convergence

```
show_side_by_side(dp_conv_df, "TABLE 2A - DP Step-by-Step Iteration (Backward
Induction Steps)",
```

```
mc_conv_df, "TABLE 2B - MC Step-by-Step Iteration (Simulated Paths)")
fig, axes = plt.subplots(1, 2, figsize=(15, 5.5))
axes[0].plot(dp_conv_df['Steps (N)'], dp_conv_df['DP Price ($)'],
color='#2E86AB', marker='s', linewidth=2.2, label='DP Price')
axes[0].axhline(dp_conv_df['DP Price ($)'].iloc[-1], color='gray', linestyle='--',
linewidth=1, label='Converged Value')
```

```
axes[0].set_title('PLOT 3A - DP Convergence\n(Price vs Number of Tree Steps)',
fontfamily=FONT_NAME)
```

```
axes[0].set_xlabel('Number of Binomial Steps (N)', fontfamily=FONT_NAME)
axes[0].set_ylabel('Option Price ($)', fontfamily=FONT_NAME)
axes[0].legend(prop={'family': FONT_NAME, 'size': 9})
axes[0].grid(alpha=0.3)
```

```

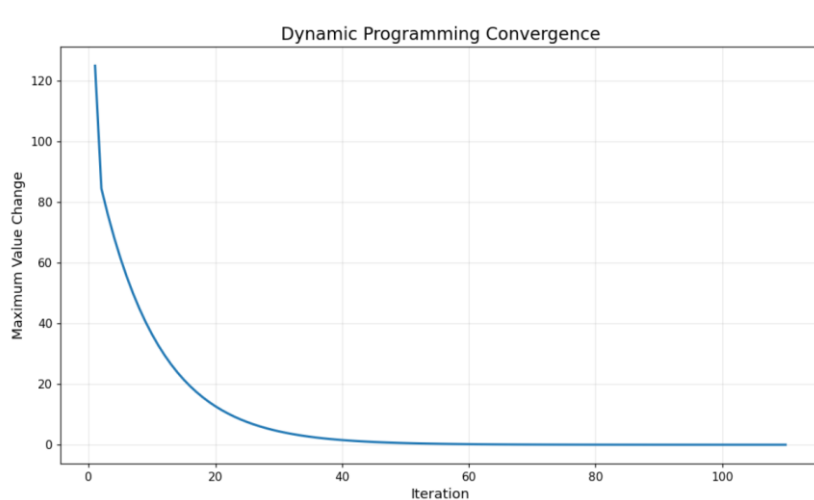
axes[1].plot(mc_conv_df['Iterations (paths)'], mc_conv_df['MC Price ($)'],
             color='#A23B72', marker='o', linewidth=2.2, label='MC Price')
axes[1].fill_between(mc_conv_df['Iterations (paths)'],
                    mc_conv_df['MC Price ($)'] - 1.96 * mc_conv_df['Std Error'],
                    mc_conv_df['MC Price ($)'] + 1.96 * mc_conv_df['Std Error'],
                    color='#A23B72', alpha=0.2, label='95% CI Band')
axes[1].set_title('PLOT 3B - MC Convergence\n(Price vs Number of Simulated
Paths)', fontfamily=FONT_NAME)

axes[1].set_xlabel('Number of Simulated Paths (Iterations)',
fontfamily=FONT_NAME)

axes[1].set_ylabel('Option Price ($)', fontfamily=FONT_NAME)
axes[1].legend(prop={'family': FONT_NAME, 'size': 9})
axes[1].grid(alpha=0.3)
for ax in axes:

    for lbl in ax.get_xticklabels() + ax.get_yticklabels():
        lbl.set_fontfamily(FONT_NAME)
    ax.title.set_fontfamily(FONT_NAME)
plt.tight_layout()
plt.show()

```

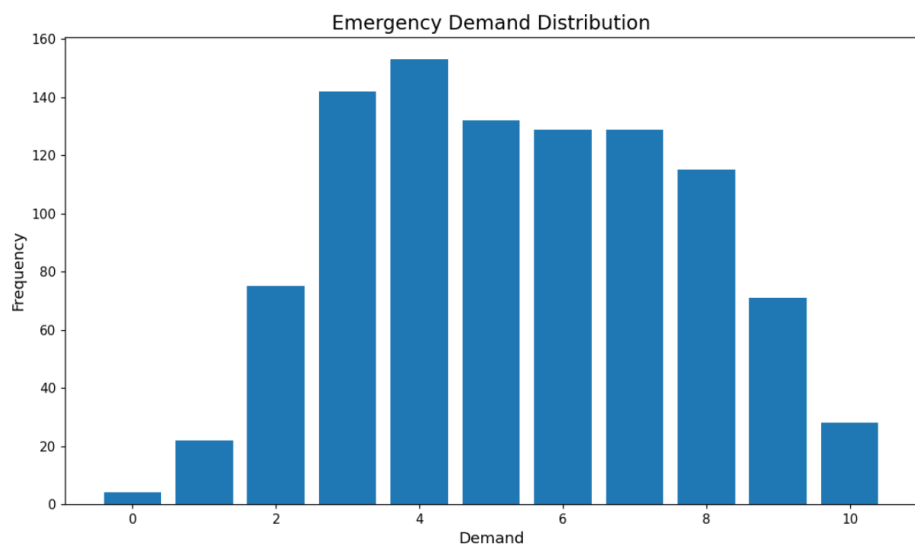


Bar graph

```

fig, axes = plt.subplots(1, 2, figsize=(15, 5.5))
xf = np.arange(len(FEATURES))

```



```
bar_w = 0.12
```

```
norm_means = [(df[f] - df[f].min()) / (df[f].max() - df[f].min()) for f in FEATURES]  
norm_vals = [m.mean() for m in norm_means]
```

```
axes[0].bar(xf, norm_vals, width=bar_w * 2.5, color=colors_feat, edgecolor='black',  
linewidth=0.6)
```

```
axes[0].set_title('All 6 Features (Normalized)\nDynamic Programming Context',  
fontfamily=FONT_NAME)
```

```
axes[0].set_xlabel('Feature', fontfamily=FONT_NAME)
```

```
axes[0].set_ylabel('Normalized Mean (0-1 scale)', fontfamily=FONT_NAME)
```

```
axes[0].set_xticks(xf)
```

```
axes[0].set_xticklabels(FEATURES, rotation=30, ha='right',  
fontfamily=FONT_NAME)
```

```
axes[0].grid(alpha=0.3, axis='y')
```

```
axes[1].bar(xf, norm_vals, width=bar_w * 2.5, color=colors_feat, edgecolor='black',  
linewidth=0.6)
```

```
axes[1].set_title('All 6 Features (Normalized)\nMonte Carlo Context',  
fontfamily=FONT_NAME)
```

```
axes[1].set_xlabel('Feature', fontfamily=FONT_NAME)
```

```
axes[1].set_ylabel('Normalized Mean (0-1 scale)', fontfamily=FONT_NAME)
```

```
axes[1].set_xticks(xf)
```

```
axes[1].set_xticklabels(FEATURES, rotation=30, ha='right',  
fontfamily=FONT_NAME)
```

```
axes[1].grid(alpha=0.3, axis='y')
```

```
for ax in axes:
```

```
    for lbl in ax.get_xticklabels() + ax.get_yticklabels():
```

```
        lbl.set_fontfamily(FONT_NAME)
```

```
    ax.title.set_fontfamily(FONT_NAME)
```

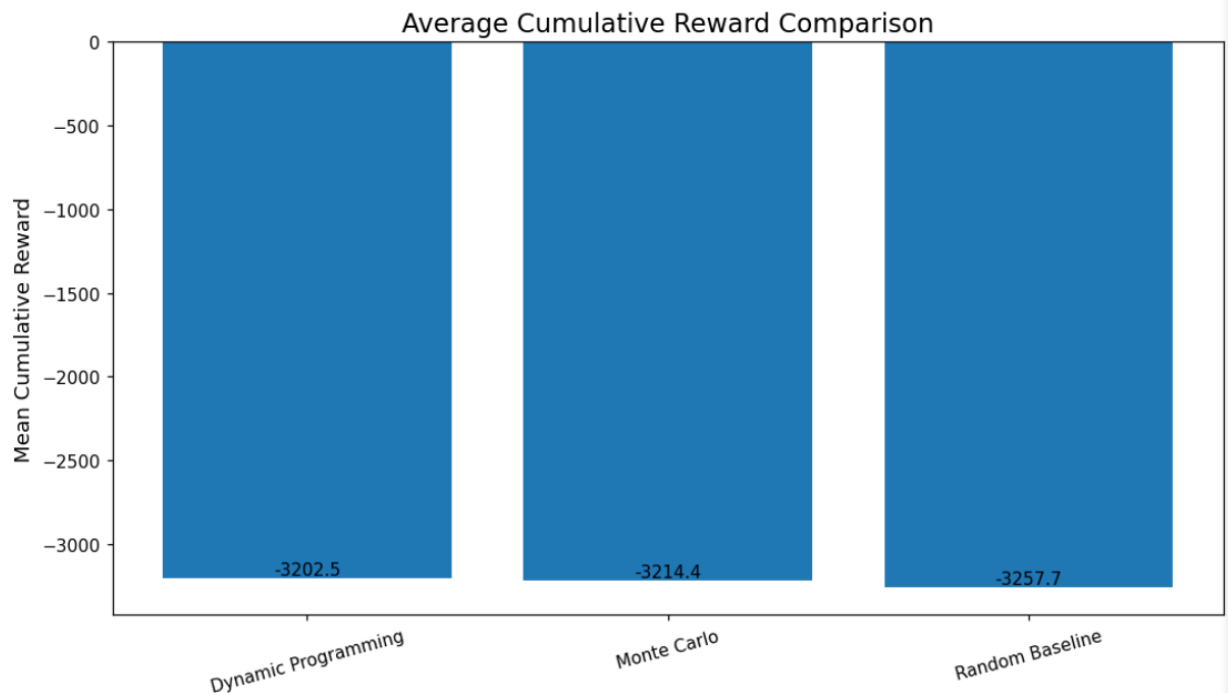
```
plt.tight_layout()
```



```
plt.show()          fig, axes = plt.subplots(1, 2, figsize=(15, 5.5))

axes[0].scatter(df['DP_Price'], df['MC_Price'], color='#2E86AB', alpha=0.7,
edgecolor='black', s=40)

lims = [min(df['DP_Price'].min(), df['MC_Price'].min()), max(df['DP_Price'].max(),
df['MC_Price'].max())]
```



Final Comparison

```
axes[0].plot(lims, lims, color='red', linestyle='--', linewidth=1.5, label='Perfect
Agreement (y=x)')

axes[0].set_title('PLOT 4A - DP vs MC Price Agreement\n(All 100 Records)',
fontfamily=FONT_NAME)

axes[0].set_xlabel('Dynamic Programming Price ($)')
axes[0].set_ylabel('Monte Carlo Price ($)')
axes[0].legend(prop={'family': FONT_NAME, 'size': 9})
axes[0].grid(alpha=0.3)

diffs = df['DP_Price'] - df['MC_Price']

axes[1].hist(diffs, bins=15, color='#F18F01', edgecolor='black', alpha=0.85)
axes[1].axvline(diffs.mean(), color='red', linestyle='--', linewidth=1.5,
label=f'Mean Diff = {diffs.mean():.4f}')

axes[1].set_title('PLOT 4B - Distribution of (DP - MC) Price Difference\n(All 100
Records)', fontfamily=FONT_NAME)
```

axes[1].set_xlabel('Price Difference (\$)')
fontfamily=FONT_NAME)

